

Combinational Circuits

Combinational Circuits

A **combinational circuit** is a circuit built by internally *combining* a number of logic gates

this lets it expose only a small number of external connections, while performing a more complex, useful function internally.

Formally:

A combinational circuit is a circuit with multiple inputs and multiple outputs, in which the output values are **uniquely and immediately determined by the current input values**

Given the same set of inputs, a combinational circuit will *always* produce the same outputs.

- There is no "history" or "memory" involved - the output at any instant depends only on the input at that same instant.

Combinational Circuits

Not all circuits behave this way.

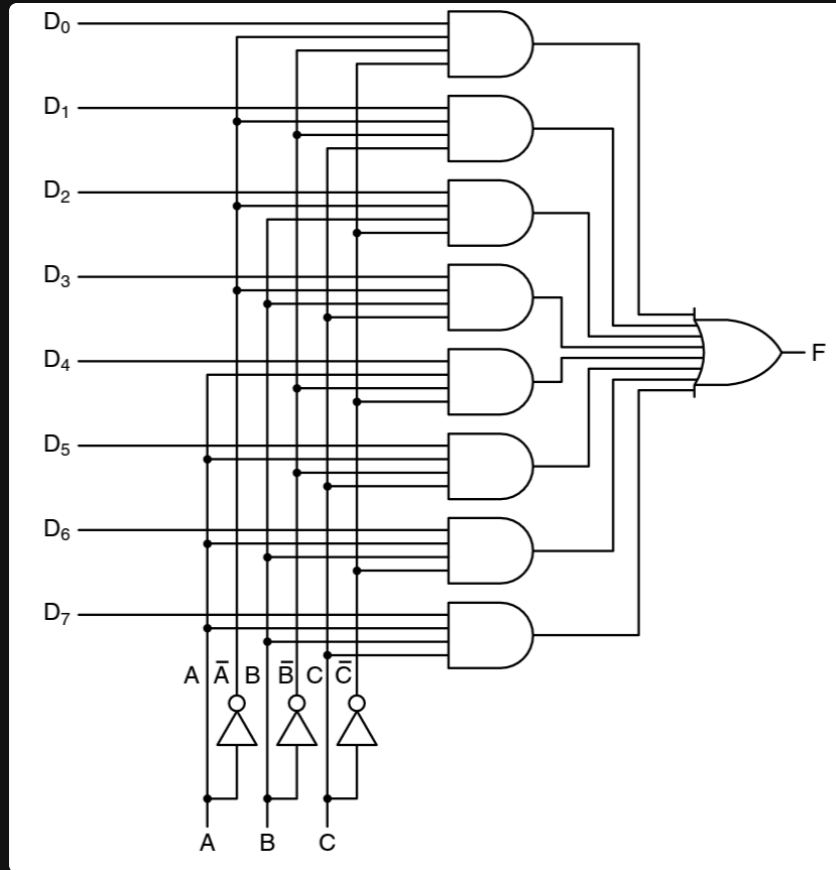
Memory circuits (and sequential circuits more generally) are the counterexample,

Their output depends **not just** on the current inputs, but also on the circuit's *previous state*.

We'll study these separately from combinational circuits.

	Combinational Circuit	Sequential/Memory Circuit
Output depends on	Current inputs only	Current inputs and past state
Has memory?	No	Yes
Example	Multiplexer, adder	Flip-flop, register

Multiplexers



A **multiplexer** is a combinational circuit that *selects* one of several input signals and forwards it to a single output.

Formally, a multiplexer is built with:

- 2^n data inputs - the pool of signals.
- n control pins - these encode which one of the 2^n inputs to pass through.
- 1 output - carries the value of whichever input was selected.

Think of it as a controllable switch:

- the n control pins act like an address, and
- whatever binary number they represent determines which of the 2^n inputs gets "*connected*" to the output.

Multiplexers

For example,

- with $n = 2$ control pins,
- there are $2^2 = 4$ data inputs, and
- the 2-bit control value (00, 01, 10, or 11) picks which of the four gets routed to the output.

Exercise

Figure out how this works (in class)

Abstraction

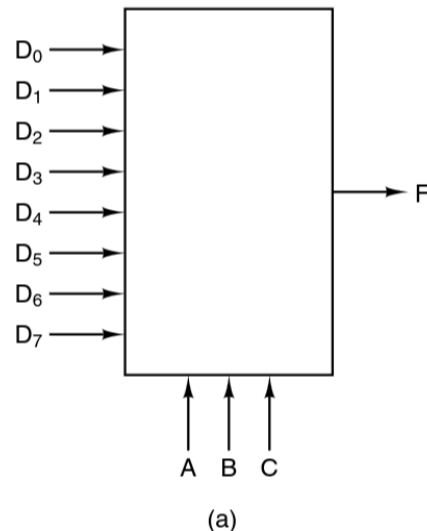
Once we've built and verified a circuit (like a multiplexer) out of individual gates,

we don't need to keep redrawing all of its internal wiring every time we use it.

Instead, we **abstract** it into a single labeled block:

- The internal gate-level detail is *hidden*.
- The block shows only what matters to someone using it: its inputs, its outputs, and its function.

This is usually called *black boxing* and for larger circuits, this is how we'll represent them



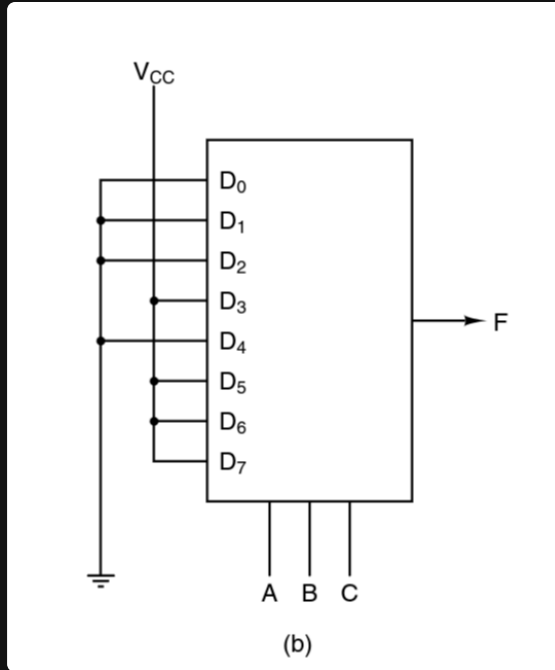
The Majority Function, Built from the MUX Abstraction

The *majority function* outputs a 1 if a majority (more than half) of its inputs are 1, and a 0 otherwise.

For 3 inputs, this means: output = 1 whenever at least 2 of the 3 inputs are 1.

We can implement the majority function without designing new gate-level logic:

- Treat the *three* inputs as the *select lines* of a MUX.
- Wire the *data inputs* of the MUX to constants like ground or live (0 or 1), based on what the correct output should be for each combination of the two select inputs.
- The resulting MUX, configured this way, reproduces the truth table of the 3-input majority function exactly.



Demultiplexers (DEMUX)

A **demultiplexer** performs the *inverse* operation of a multiplexer.

formally, a demultiplexer is built with:

- 1 input - the single signal to be routed.
- n control (select) pins - encode, in binary, which output the input should be sent to.
- 2^n outputs - only one of which receives the input signal at any given time

the rest stay inactive.

imagine a single signal source and a bank of light bulbs.

The demultiplexer is like a *switch* that decides *which one* light bulb turns on, based on the control setting the "electricity" (the single input) is *routed* to exactly one destination out of many possible ones.

Where a MUX answers "which of many inputs do I let through?", a DEMUX answers "which of many outputs do I send this one input to?"

Exercise

Make a demultiplexer (in class)

Real-World Example: Keyboard Wiring

A standard keyboard has **104 keys**.

Wiring each key with its own dedicated pair of connections to the controller chip would be wasteful and impractical. Instead, keyboards use a *matrix layout*:

- Keys are arranged in an **8 × 13 grid**:

$$8 \times 13 = 104 \text{ keys}$$

- Instead of 104+ individual wires, the grid only needs:

$$8 + 13 = 21 \text{ wires}$$

The controller scans the grid row by row (or column by column)

effectively using *demultiplexing* logic to select a row/column and multiplexing logic to read back which key(s) are pressed at each intersection.

This is a direct, practical application of the MUX/DEMUX concepts above:

- a small number of control lines (21) can address a much larger number of individual points (104) by exploiting the 2D grid structure,

rather than needing one dedicated wire per key.

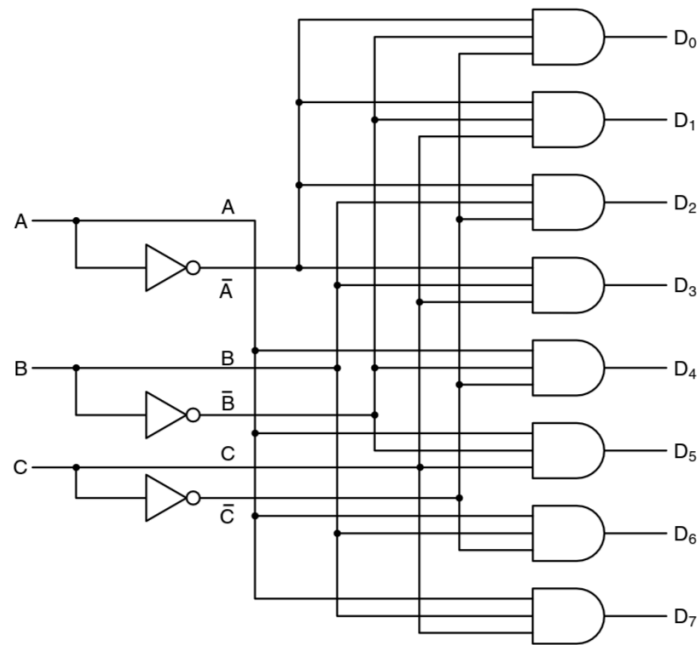
Decoders

Decoders

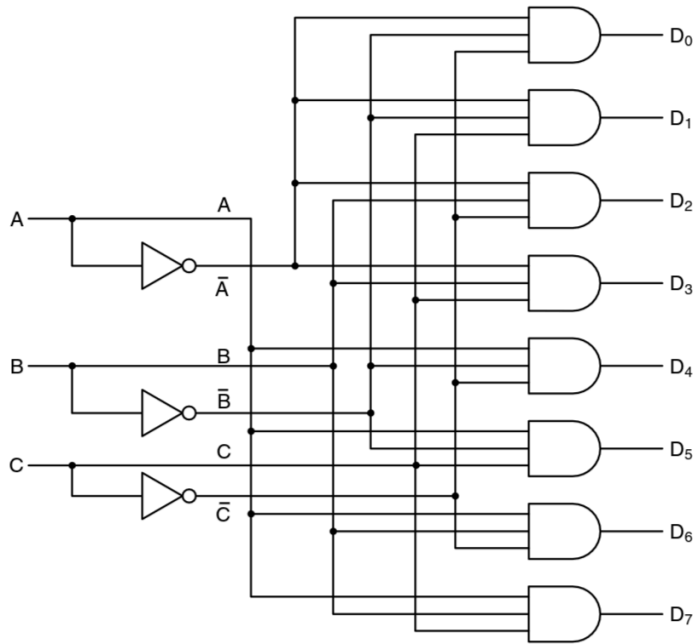
Another example of a combinational circuit is a **decoder**

A decoder takes in an

- n -bit number as input and
- uses it to *select* (i.e., set to 1) exactly one of the 2^n -output lines



Decoders



Imagine a small memory consisting of **eight** chips, each containing **256 MB**.

1. Chip 0 has addresses 0 to 256 MB,
2. Chip 1 has addresses 256 MB to 512 MB,
3. and so on.

When an address is presented to the memory, the high-order 3 bits are used to **select** one of the eight chips.

Decoders

Using the circuit, these 3 bits are the three inputs, A, B, and C.

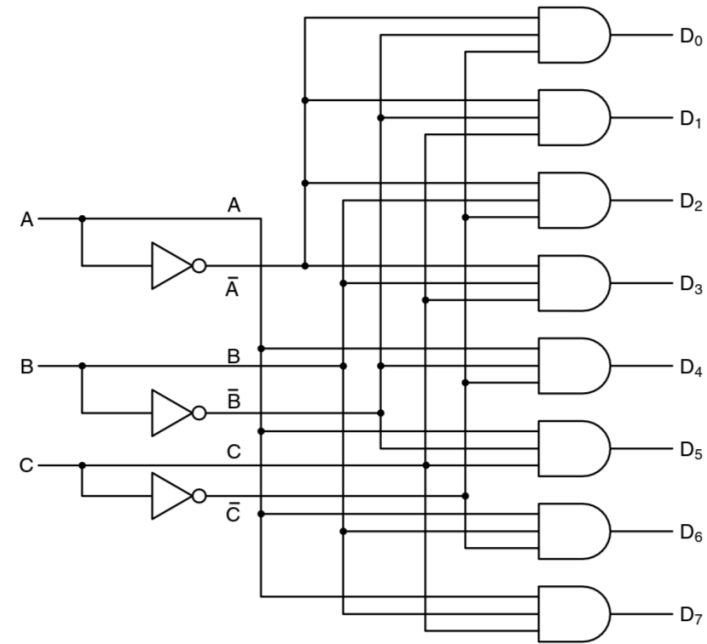
Depending on the inputs, **exactly one** of the eight output lines, D0 , . . . , D7 , is 1;

The rest are 0.

Each output line *enables one of the eight* memory chips.

Because only one output line is set to 1, **only one chip is enabled**

And so you can use the rest of the bits to now *address* the 256 Mb memory lines inside that enabled chip



Decoders

Without a decoder, every chip would need its own *equality check* on the 3 high-order address bits:

- Chip 0 enabled when `ABC = 000`
- Chip 1 enabled when `ABC = 001`
- ...
- Chip 7 enabled when `ABC = 111`

That's **8 separate comparator circuits**, all testing the same 3 bits in parallel.

With a decoder, a single shared 3-to-8 circuit replaces all eight:

- Produces **all 8 enable lines at once**
- **One-hot** output — exactly one chip can ever be enabled
- **Scales cheaply**: n input bits give 2^n outputs, so 10 bits could address **1024 chips** with one small circuit

Comparators

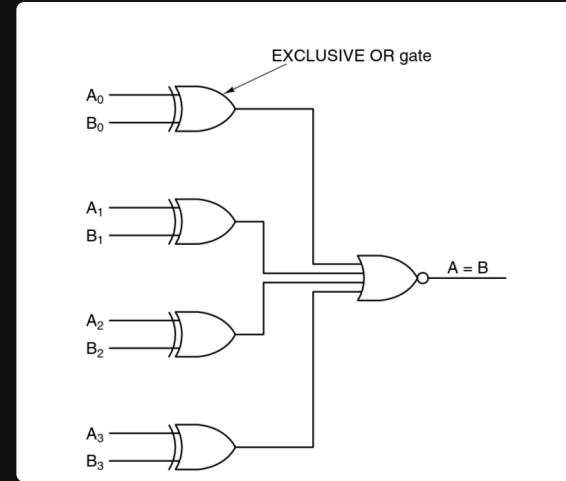
Comparators

A **comparator** is another useful circuit that *compares* two input words.

The simple comparator takes two inputs, **A** and **B**, each of length **4 bits**, and produces a **1** if they are *equal* and a **0** otherwise.

It is built from **XOR** (EXCLUSIVE OR) gates:

- XOR outputs **0** if its inputs are *equal*
- XOR outputs **1** if its inputs *differ*



If the two words are equal, **all four** XOR gates output 0.

A **NOR** gate is used as the final stage to *reverse* the sense of the test: **1** means equal, **0** means unequal.